

SQL Query Optimization

How to find the slow query, understand why it's slow, and fix it. Every example targets MySQL 8 with InnoDB.

Sample tables: customers (id, name, email) and orders (id, customer_id, status, total, created_at).

1 Read the plan with EXPLAIN

Before you change anything, ask MySQL how it plans to run the query.

```
EXPLAIN
SELECT id, total FROM orders
WHERE customer_id = 42;
```

COLUMN	WHAT TO LOOK FOR
type	How it reads the table (see scale below)
key	Index it uses. NULL = none
rows	Rows it expects to examine
Extra	Using filesort or Using temporary = warning

ALL

index

range

ref

eq_ref

const

Worst: scans the whole table

Best: a single row

EXPLAIN ANALYZE (8.0.18 or later) runs the query and shows real timings, not estimates.

3 Composite indexes: the leftmost prefix

A multi-column index works left to right, like a phone book sorted by last name, then first name.

```
CREATE INDEX idx_customer_status_date
ON orders (customer_id, status, created_at);
```

WHERE	USES THE INDEX?
customer_id = 42	Yes
customer_id = 42 AND status = 'paid'	Yes
status = 'paid'	Rarely

Recommended order: equality (=) columns first, the range column (>, <, BETWEEN) last.

2 Index where you filter and join

Columns in WHERE, JOIN ... ON and ORDER BY are your index candidates.

```
CREATE INDEX idx_orders_customer
ON orders (customer_id);

-- List a table's indexes
SHOW INDEX FROM orders;
```

Every index slows down INSERT and UPDATE and uses disk. Create the ones your queries use, not one per column.

4 Don't hide the column inside a function

When the indexed column sits inside a function or a calculation, MySQL can't use the index and checks row by row.

AVOID

```
WHERE YEAR(created_at) = 2025
```

BETTER

```
WHERE created_at >= '2025-01-01'
AND created_at < '2026-01-01'
```

AVOID

```
WHERE email LIKE '%@gmail.com'
```

BETTER

```
WHERE email LIKE 'ana%'
```

Mixed types do the same: if phone is VARCHAR, write phone = '7025550100' with quotes.

5 Select only the columns you use

SELECT * pulls data your code never reads. If every requested column is in the index, MySQL doesn't even open the table: EXPLAIN shows Using index under Extra.

AVOID

```
SELECT * FROM orders WHERE customer_id = 42;
```

BETTER

```
-- With INDEX (customer_id, status, total)
SELECT status, total
FROM orders
WHERE customer_id = 42;
```

That's called a covering index.

7 Avoid the N+1 problem

One query for the list, then one more per row inside a loop. With 200 customers that's 201 trips to the database.

AVOID

```
-- once
SELECT id, name FROM customers;
-- once PER customer, inside the PHP loop
SELECT COUNT(*) FROM orders WHERE customer_id = ?;
```

BETTER

```
SELECT c.id, c.name, COUNT(o.id) AS orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.id, c.name;
```

9 Review before shipping to production

- ☐ Did you run EXPLAIN on every new query?
- ☐ Does any large table show type = ALL?
- ☐ Is there a function or calculation on a WHERE column?
- ☐ Is there a SELECT * you could trim?
- ☐ Is there a query inside a PHP loop?
- ☐ Does any pagination use an OFFSET that can grow without limit?

```
-- Indexes nobody has used since the last server restart
SELECT * FROM sys.schema_unused_indexes
WHERE object_schema = 'your_database';
```

6 Paginate with keyset, not OFFSET

With OFFSET, MySQL reads every earlier row and then throws it away. On page 5,000 that's 100,000 wasted rows.

AVOID

```
SELECT id, total FROM orders
ORDER BY id DESC
LIMIT 20 OFFSET 100000;
```

BETTER

```
SELECT id, total FROM orders
WHERE id < 250310 -- last id shown
ORDER BY id DESC
LIMIT 20;
```

The tradeoff: you can't jump straight to page 37. It fits "load more" and infinite scroll.

8 To check existence, don't count

COUNT(*) walks every matching row. EXISTS stops at the first one.

AVOID

```
SELECT COUNT(*) FROM orders
WHERE customer_id = 42; -- then: if ($n > 0)
```

BETTER

```
SELECT EXISTS (
  SELECT 1 FROM orders WHERE customer_id = 42
) AS has_orders;
```

Returns 1 or 0.